

REPORTE PRACTICA S05

OBJETIVO

El objetivo principal de esta práctica es que desplaguemos y probemos localmente la aplicación monolítica "Mexi Banco" utilizando Docker Compose, con la finalidad de entender su estructura actual antes de comenzar a dividirla en futuras sesiones. Para ello realizaremos transacciones y movimientos en las cuentas, así como algunos casos donde se presenta otra situación/error

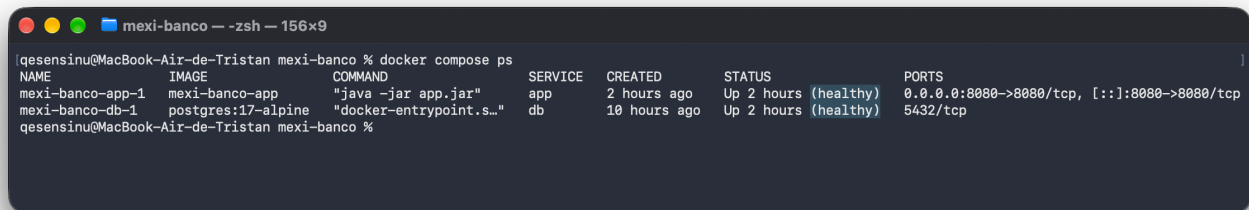
DESARROLLO

- Paso 1

Para iniciar con esta practica primero debemos tener descargada la aplicación de "Postman" de escritorio, descargar la colección de Postman y clonar el repositorio (`git clone --branch v05.1 https://github.com/OscarRuiz21/mexi-banco.git`) y entrar a esa carpeta.

- Paso 2

Una vez clonado el repositorio, procederemos a levantar un docker compose con el comando: `docker compose up --build -d`. Comprobaremos que el repositorio este correctamente levantado con: `docker compose ps`



```
mexi-banco -- zsh -- 156x9
| qesensinu@MacBook-Air-de-Tristan mexi-banco % docker compose ps
|
| NAME                IMAGE                COMMAND                SERVICE  CREATED      STATUS      PORTS
| mexi-banco-app-1    mexi-banco-app      "java -jar app.jar"    app       2 hours ago  Up 2 hours (healthy)  0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
| mexi-banco-db-1     postgres:17-alpine  "docker-entrypoint.s..." db        10 hours ago Up 2 hours (healthy)  5432/tcp
| qesensinu@MacBook-Air-de-Tristan mexi-banco %
```

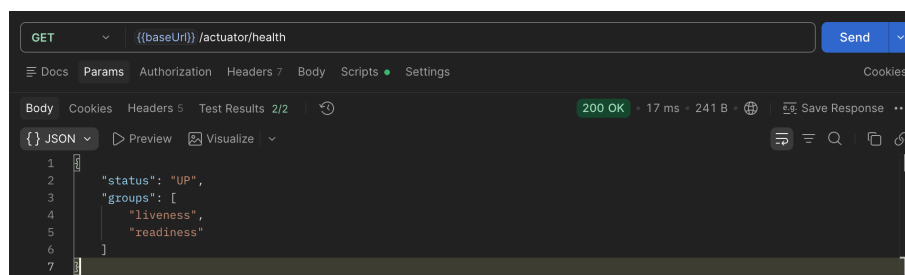
Sabremos que se levanto correctamente por que el status sera **healthy**.

- Paso 3

Para este paso probaremos los endpoints de Mexi Banco con ayuda de Postman. Para ello es necesario importar la colección de Postman descargada anteriormente. Una vez dentro empezaremos a probar cada endpoint:

○ Abrir cuentas

Primero probaremos la salud/estado del sistema:

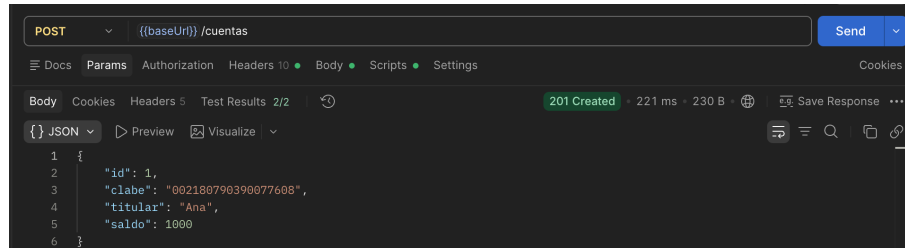


```
GET {{baseUrl}}/actuator/health
Send

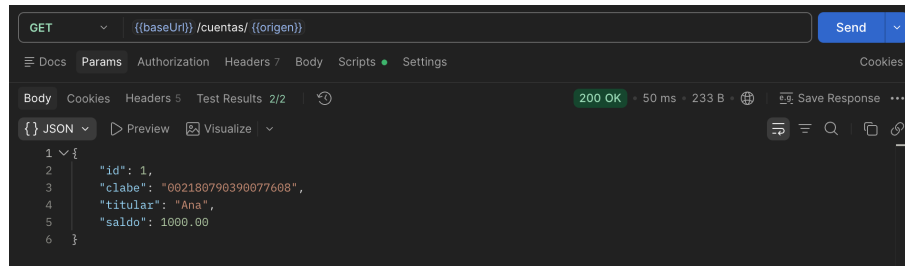
Body
Cookies
Headers 5
Test Results 2/2
200 OK • 17 ms • 241 B
Save Response

{
  "status": "UP",
  "groups": [
    "liveness",
    "readiness"
  ]
}
```

Despues abriremos una cuenta que etiquetaremos como cuenta de origen, con “Ana” como titular y un saldo de 1000:

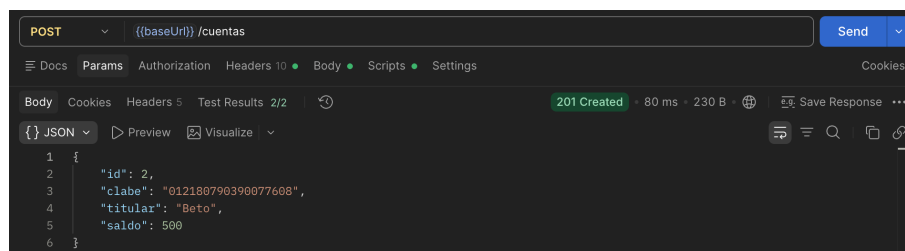


Por ultimo y a modo de verificacion, consultaremos la cuenta de origen que hemos creado:

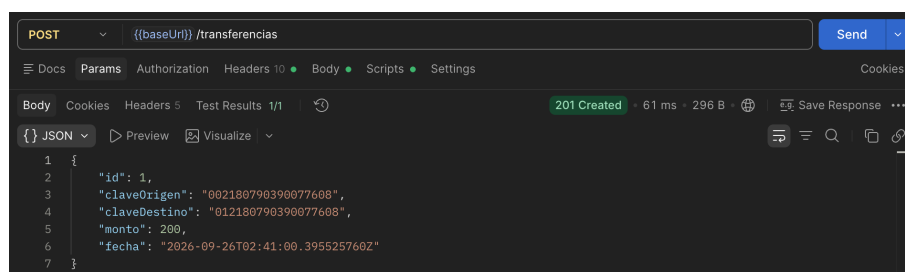


○ Transferencias

Ahora realizaremos una transferencia, por lo que es necesario crear una cuenta de destino, con “Beto” como titular y un saldo de 500:



Seguido realizaremos la transferencia de la cuenta de origen a la cuenta de destino (De Ana a Beto) con un monto de 200:



Por ultimo corroboraremos el estado de las cuentas, las cuales tuvieron que sufrir cambios en sus saldos, Ana deberia de terminar con 800 y Beto con 700:

```
GET {{baseUrl}} /cuentas/{{origen}}

Body
Cookies
Headers 5
Test Results 2/2
Scripts
Settings
Cookies

200 OK • 19 ms • 232 B • Save Response ...

JSON Preview Visualize
1 {
2   "id": 1,
3   "clabe": "002180790390077608",
4   "titular": "Ana",
5   "saldo": 800.00
6 }
```

```
GET {{baseUrl}} /cuentas/{{destino}}

Body
Cookies
Headers 5
Test Results 2/2
Scripts
Settings
Cookies

200 OK • 16 ms • 233 B • Save Response ...

JSON Preview Visualize
1 {
2   "id": 2,
3   "clabe": "002180790390077608",
4   "titular": "Beto",
5   "saldo": 700.00
6 }
```

Como añadido tambien consultaremos la transferencia:

```
GET {{baseUrl}} /transferencias/{{transferenciald}}

Body
Cookies
Headers 5
Test Results 2/2
Scripts
Settings
Cookies

200 OK • 36 ms • 296 B • Save Response ...

JSON Preview Visualize
1 {
2   "id": 1,
3   "claveOrigen": "002180790390077608",
4   "claveDestino": "012180790390077608",
5   "monto": 200.00,
6   "fecha": "2026-09-26T02:41:00.395526Z"
7 }
```

○ Idempotencia

Como sabemos, la idempotencia es que al realizar una accion mas de una vez esta tenga el mismo resultado. Este concepto se aplica en las transferencias, por ejemplo, si realizamos un pago a una cuenta con un id específico, al realizar el proceso de este mismo pago, con el mismo id, el resultado sera el cobro de uno solo de estos. Para explicarlo de una mejor forma, primero realizaremos dicho pago una vez, quitaremos 50 a la cuenta origen (la cual se habia quedado con 800) por lo que esta quedaria con un saldo de 750:

```
POST {{baseUrl}} /spei

Body
Cookies
Headers 5
Test Results 2/2
Scripts
Settings
Cookies

201 Created • 479 ms • 334 B • Save Response ...

JSON Preview Visualize
1 {
2   "id": 1,
3   "idempotencyKey": "prueba-790390077608",
4   "claveOrigen": "002180790390077608",
5   "bancoDestino": "BBVA",
6   "claveDestino": "012180000000000099",
7   "monto": 50,
8   "estado": "ENVIADO"
9 }
```

Como vemos, el id de este pago es el '1'. Si realizamos el mismo pago, el saldo de la cuenta deberia de mantenerse en 750, ya que se esta realizando el mismo pago, no uno nuevo, el mismo:

POST {{baseUrl}} /spei

201 Created · 70 ms · 337 B

```
1 {
2   "id": 1,
3   "idempotencyKey": "prueba-796390077608",
4   "claveOrigen": "002180790390077608",
5   "bancoDestino": "BBVA",
6   "claveDestino": "012180000000000099",
7   "monto": 50.00,
8   "estado": "ENVIADO"
9 }
```

Si vemos el estado de la cuenta origen, veremos el saldo de 750, como lo habia comentado anteriormente:

GET {{baseUrl}} /cuentas/{{origen}}

200 OK · 13 ms · 232 B

```
1 {
2   "id": 1,
3   "clave": "002180790390077608",
4   "titular": "Ana",
5   "saldo": 750.00
6 }
```

○ Estados de las cuentas

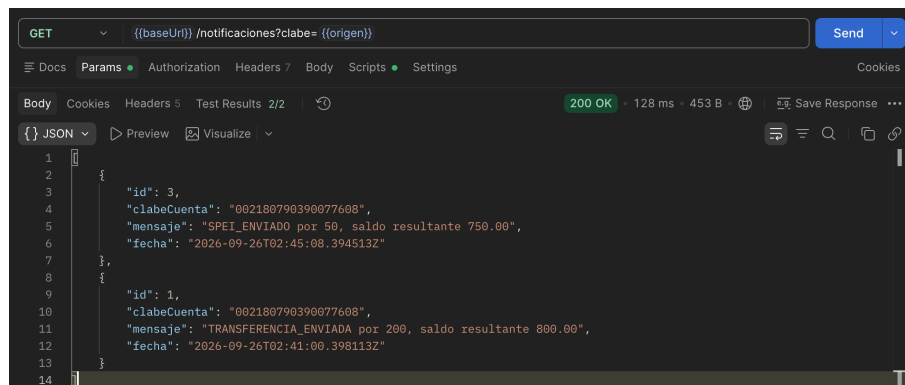
Ahora revisaremos los movimientos de las cuentas, empezando con la cuenta origen (Ana), donde veremos la “transferencia” inicial de 1000, luego los cobros de 200 y 50:

GET {{baseUrl}} /movimientos?clave={{origen}}

200 OK · 37 ms · 712 B

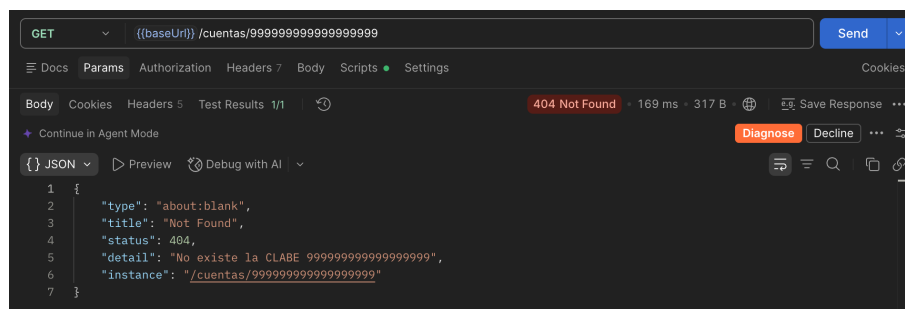
```
1 [
2   {
3     "id": 5,
4     "claveCuenta": "002180790390077608",
5     "tipo": "SPEI_ENVIADO",
6     "monto": -50.00,
7     "saldoResultante": 750.00,
8     "fecha": "2026-09-26T02:45:07.974563Z",
9     "referencia": "BBVA · 012180000000000099"
10  },
11  {
12    "id": 3,
13    "claveCuenta": "002180790390077608",
14    "tipo": "TRANSFERENCIA_ENVIADA",
15    "monto": -200.00,
16    "saldoResultante": 800.00,
17    "fecha": "2026-09-26T02:41:00.392847Z",
18    "referencia": "a 012180790390077608"
19  },
20  {
21    "id": 1,
22    "claveCuenta": "002180790390077608",
23    "tipo": "DEPOSITO",
24    "monto": 1000.00,
25    "saldoResultante": 1000.00,
26    "fecha": "2026-09-26T02:34:37.839711Z",
27    "referencia": "saldo inicial"
28  }
29 ]
```

Tambien podemos ver las “notificaciones”, donde se anuncian los movimientos realizados (el deposito inicial no se mostrara):

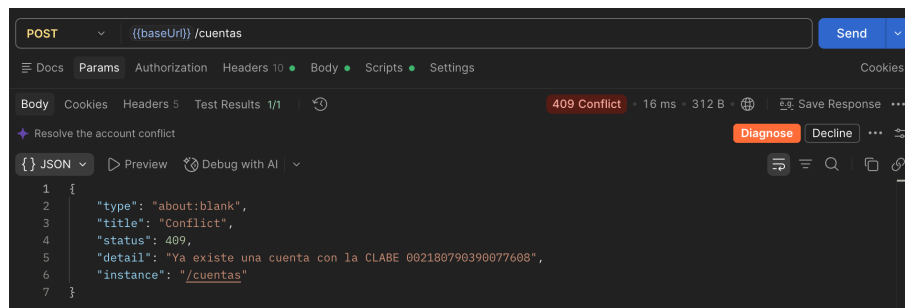


○ Avisos y errores

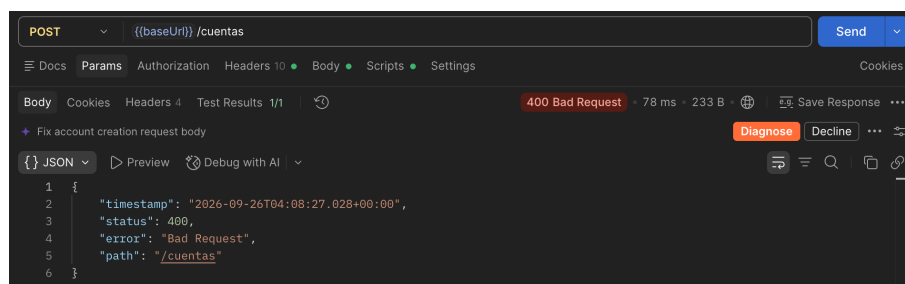
Por ultimo, Mexi Banco tambien cuenta con codigos de errores o avisos. Por ejemplo, de una cuenta que no existe:



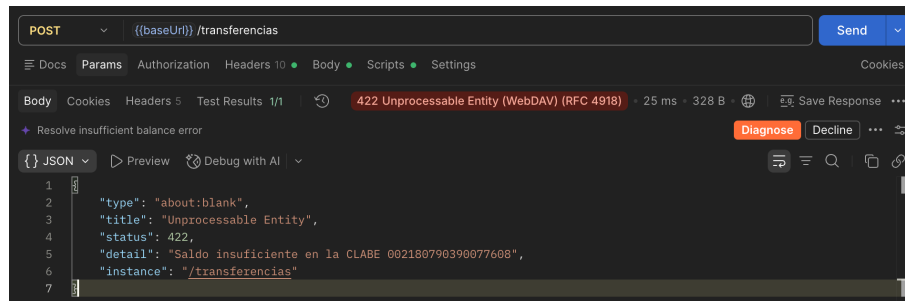
De una CLABE que se repite:



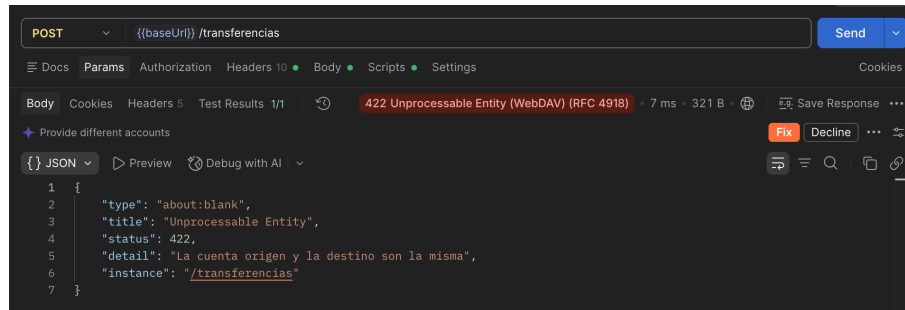
De una cuenta sin clabe:



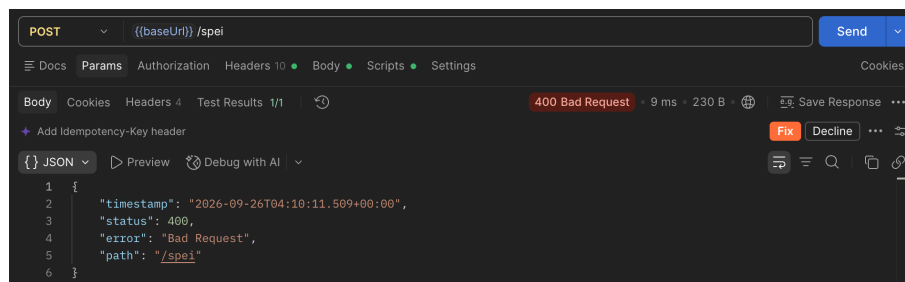
Aviso de saldo insuficiente:



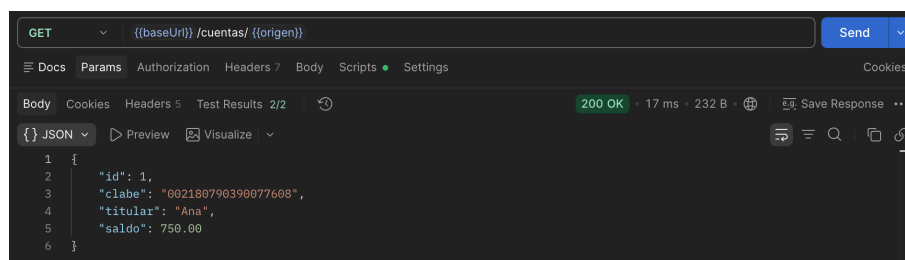
Transferencia a la misma cuenta:



SPEI sin idempotencia:

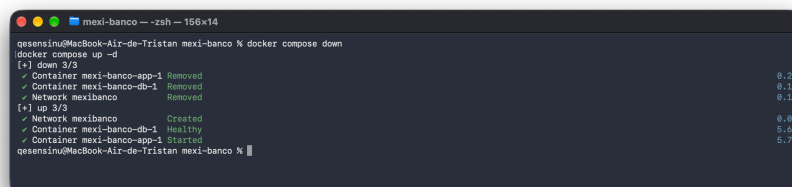


Al ejecutar los errores anteriores no se movio el saldo de la cuenta, lo cual es correcto para tener consistencia en el sistema:

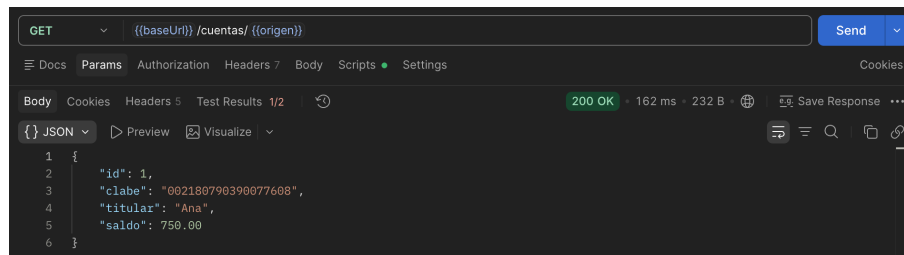


- PASO 4

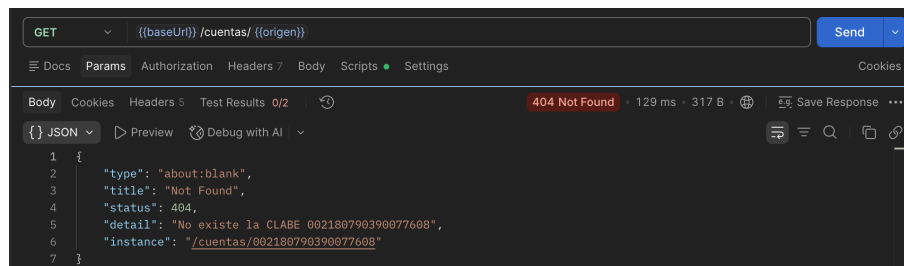
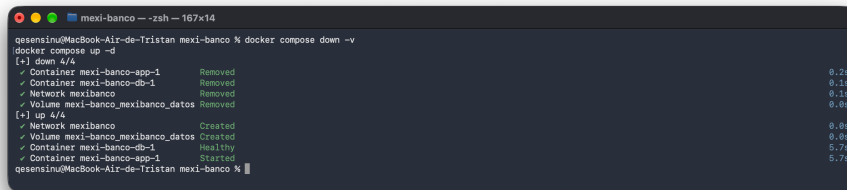
Para este paso bajaremos el docker compose con y sin volumenes. Primero lo apagaremos **con volumen**, como sabemos, al realizar esto, los datos que se registraron en el docker se mantendran:



Ahora consultaremos la cuenta, la cual tenia un saldo de 750:



Ahora, al hacerlo sin volumen, lo que esperamos es que la cuenta se borre:



CONCLUSION

Esta práctica me permitió entender el despliegue y validación de una aplicación monolítica usando contenedores, para así poder familiarizarnos con dicha aplicación. Al finalizar la práctica si logre familiarizarme con la app de “Mexi Banco” para comprender como funcionan los procesos de cambios, quizá de una forma básica pero suficiente para entender la estructura de este negocio.

Además, Postman sirvió no solo para confirmar el correcto funcionamiento de la lógica de negocio sino también para documentar el comportamiento de la API en un entorno local e incluso pudiendo observar como interactúa cada endpoint y como afectan cada uno según el proceso, así como la importancia de definir cada uno para una mejor claridad en la estructura de este sistema.